FractalFlow: Portable Deep-Zoom Julia Rendering with Perturbation Theory across WebGPU and CUDA

Timofei Amosov

Technical Report, Version 1.1.0 – 29 July 2026

doi:10.5281/zenodo.21686326

Abstract

FractalFlow renders deep zooms of the quadratic Julia set with perturbation: one reference orbit per view, iterated on the CPU in double-double arithmetic, and a per-pixel deviation from that orbit iterated on the GPU at low precision. The same recurrence runs in a WebGPU/WGSL fragment shader with binary32 deltas and in a CUDA kernel with binary64 deltas; a WebGL2 direct-iteration path is kept for compatibility only. One implementation detail decides how deep either backend can go. Uploading the reference orbit in absolute coordinates puts the rebasing subtraction $z - Z_0$ on the grid of order-one values in the upload format, which flattens pixel offsets from about 10^{-5} view scale downwards in binary32 and from about 10^{-13} in binary64; uploading $Z_m - Z_0$, formed in double-double before the rounding, removes the failure in both. The effect is not subtle: at a view scale of 10^{-16} the absolute encoding misses 3509 of 4096 pixels, the relative one reproduces the arbitrary-precision reference exactly. Six fixed 64×64 views are compared pixel by pixel against direct `mpmath` iteration at 50–90 digits, two of them below what binary64 pixel coordinates can address; a depth sweep past the last of them locates where the pipeline stops agreeing, and the camera floor is set there rather than at the representation's theoretical reach. Throughput on one RTX 3060 Ti peaks at 1168 GIter/s for WebGPU and 70.8 GIter/s for CUDA. Those rates charge every pixel the full iteration cap; measured escape times show the hardware issuing about 7% of that at the peak view, so the figure is a fixed-workload normalizer rather than a count of executed work.

1 Motivation and contribution

For the quadratic map

$$f_c(z) = z^2 + c, \qquad z, c \in \mathbb{C}, \tag{1}$$

the filled Julia set collects the initial values whose forward orbit stays bounded, and the Julia set is its boundary [1]. An escape-time image runs that orbit for every pixel and colours the pixel by how long it survives. On a GPU this is one short loop per fragment, which is why the shallow case is easy.

The method breaks at depth, and the cause is the coordinate format rather than the compute budget. In a viewport of H rows showing a view of height s, adjacent pixel centres are s/H apart in the complex plane. Near $|z| \approx 1$, consecutive binary32 values are 6×10^{-8} to 1.2×10^{-7} apart, and consecutive binary64 values 1.1 to 2.2×10^{-16} apart. At $H = 1080$, direct iteration therefore stops separating adjacent pixels below a view height of about 10^{-4} in binary32, and about 2×10^{-13} in binary64. Adding iterations or cores changes nothing, because the two neighbours have become the same number.

Perturbation splits the work by sensitivity. One reference orbit is evaluated at high precision, and neighbouring pixels only track a small deviation from it [2, 3]. FractalFlow applies that established idea to an interactive Julia viewer, and the engineering contributions are:

1. one recurrence, two independent implementations: a WGSL fragment shader with binary32 deltas and a CUDA kernel with binary64 deltas, sharing the pixel mapping, escape radius and colour function with the Python reference renderer;
2. a double-double camera and reference orbit, with the per-pixel loop left in the GPU's native precision;
3. the reference orbit uploaded relative to Z_0 in both backends. Section 3.2 shows why the absolute form quantizes the rebasing step, at which scale it does so for each precision, and what it costs to fix;
4. a measured depth limit. The camera stops at the deepest scale that still agrees with arbitrary precision, and the sweep past it is archived rather than argued; and
5. evidence produced by scripts that share no code with the renderers: six validation cases against `mpmath`, a browser-side comparison of the WebGPU output against the same references, benchmark and escape-time measurements, a recorded environment, and a manifest of input and output hashes for every figure.

Perturbation, double-double arithmetic and rebasing are all prior work, cited below. What this report adds is the portable implementation, one precision failure at the boundary between those techniques, and an evidence package small enough for a reader to re-run.

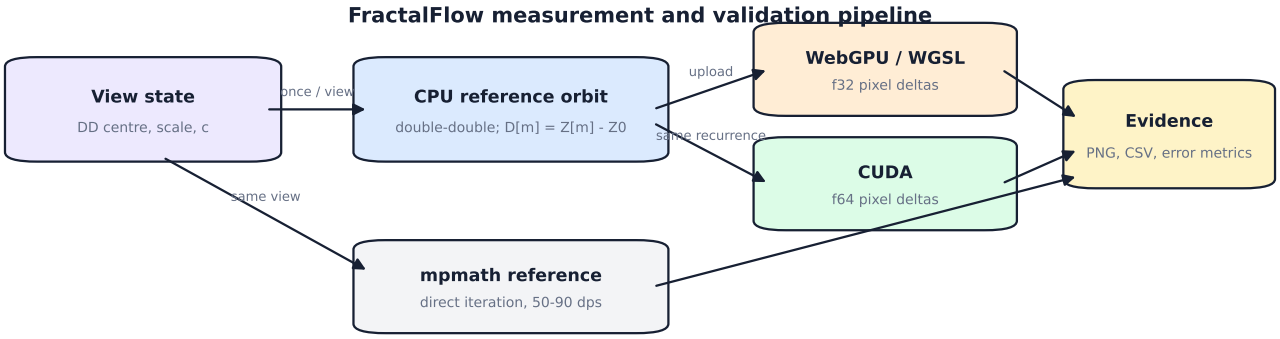

Figure 1: The precision-sensitive reference is computed once on the CPU. WebGPU and CUDA then evaluate the same per-pixel recurrence at different precisions. Direct arbitrary-precision iteration supplies an independent reference rather than reusing the implementation under test.

2 Perturbation for a Julia map

2.1 Reference and delta recurrence

Let Z_n be the reference orbit starting at the view centre Z_0:

$$Z_{n+1} = Z_n^2 + c. \tag{2}$$

A pixel starts at $z_0 = Z_0 + \delta_0$, where δ_0 is its screen-space offset after the scale and aspect-ratio mapping. Writing $z_n = Z_n + \delta_n$ and substituting into Equation 1 gives

$$\begin{aligned} Z_{n+1} + \delta_{n+1} &= (Z_n + \delta_n)^2 + c, \\ \delta_{n+1} &= 2Z_n\delta_n + \delta_n^2. \end{aligned} \tag{3}$$

The Mandelbrot derivation carries an extra δc term; here c is constant across the image and the initial value varies per pixel instead, so it drops out. Once Z_n is known, Equation 3 needs nothing but ordinary complex products and sums.

The orbit is capped at 4000 points, the same constant that caps the iteration count (`MAX_ITER_LIMIT` in the shared config). At two `f32` per point the GPU storage buffer never exceeds 32 KiB, so it stays in cache during the render. Each pixel carries a reference index m, an iteration counter and the complex delta. The escape test is applied to $z = Z_m + \delta$; escaped pixels leave the loop immediately, interior pixels run to the cap. Smooth colouring happens after the escape decision, from the same formula and the same palette stops transcribed into WGSL, CUDA and Python.

2.2 Rebasing

A delta can stop being small relative to its reference, and a finite stored orbit eventually runs out. FractalFlow uses the rebasing rule credited to Zhuoran and documented by Heiland-Allen [4]. After each delta advance, the code forms the displacement of the pixel from the first reference point,

$$\delta^{(0)} = z - Z_0 = D_m + \delta, \tag{4}$$

and restarts at $m = 0$ with $\delta \leftarrow \delta^{(0)}$ when $|\delta^{(0)}| < |\delta|$, or when m has reached the last stored sample.

In the shader that is five lines, and every one of them has a reason:

```
let D2 = refOrbit[min(m, u.refLength - 1u)];
let fx = D2.x + d.x;
let fy = D2.y + d.y;
if ((fx*fx + fy*fy) < (d.x*d.x + d.y*d.y) || m >= u.refLength - 1u) {
  d = vec2f(fx, fy); m = 0u;
}
```

The displacement is formed by adding two stored offsets, never by subtracting Z_0 from an absolute coordinate – Section 3.2 is about that choice. The subtraction is from Z_0 and not from zero: the two agree only when $Z_0 = 0$, which is exactly why a Mandelbrot-style reset $\delta \leftarrow z$ survives testing at the origin and then fails on any off-centre Julia view. And the index is clamped because a reference that escapes on its first step leaves m at the orbit length here, where an unclamped read would return whatever the buffer held.

3 Precision design

3.1 Double-double camera and orbit

A double-double number represents a value as the unevaluated sum $x_{\mathrm{hi}} + x_{\mathrm{lo}}$ of two binary64 values. Error-free transformations such as TwoSum, QuickTwoSum and Dekker's split product recover the rounding residual of the elementary operations [5–7]. The representation carries about 106 significand bits, near 31 decimal digits, and keeps the binary64 exponent range.

The browser implements addition, subtraction and multiplication only; nothing in the camera or the orbit needs a division. The CUDA host adds one, used solely to parse a high-precision centre given on the command line as a decimal string. The camera centre stays double-double through cursor-anchored zoom and pan, and the CPU evaluates Equation 2 in the same arithmetic.

The browser does not emulate that arithmetic per pixel. Doing so would trade a few thousand CPU operations for a few million compensated shader operations per frame. The reference is rounded once on upload, and the pixel delta stays a native `f32`. At the camera's floor of 10^{-20} the initial delta is still an ordinary `f32` – only its exponent has moved, and normal `f32` values reach down to 10^{-38}. What bounds the depth is the reference orbit, not the delta.

3.2 Why the uploaded orbit is relative

An early version of the shader uploaded absolute orbit samples $\mathrm{fl}_{32}(Z_m)$. That is enough to multiply a small delta by an order-one Z_m in Equation 3, and it is wrong for Equation 4. Forming $\mathrm{fl}_{32}(Z_m) + \delta - Z_0$ subtracts two order-one values that have already been rounded onto a grid of spacing near 10^{-7}. Pixel offsets much smaller than that grid do not survive the subtraction: they collapse onto the same grid point, neighbouring pixels receive identical deltas, and the image breaks into blocks. In practice this became visible from about 10^{-5} view scale and got worse with depth.

Both backends now upload

$$D_m = \mathrm{fl}(Z_m - Z_0), \tag{5}$$

with the subtraction performed in double-double on the CPU and one rounding at the end. The rebased delta is then $D_m + \delta$, a sum of two offsets that are both small exactly when their sum needs relative precision. The kernel still reconstructs an approximate Z_m as $\mathrm{fl}(Z_0) + D_m$ for the recurrence, where an absolute error at the last bit of an order-one value is harmless, but the precision-critical subtraction between two rounded order-one samples is gone.

The same failure in binary64. The CUDA kernel kept absolute samples until this release, and the argument above applies to it verbatim with a grid near 2×10^{-16} instead of 10^{-7}: at 1080 rows the quantization sets in around a view scale of 10^{-13}. Nothing in the previous report crossed that line – the benchmark stops at 3×10^{-12} and the deepest validation case was 10^{-10} – so the defect was invisible to the evidence, which is exactly what makes it worth reporting. Rendering the fixed-point case at 10^{-16} separates the two encodings without ambiguity (Figure 3). With absolute samples the frame comes out as a single colour: every pixel offset falls below the grid, every delta lands on the same value, and 3509 of 4096 pixels disagree with the arbitrary-precision reference at a mean of 128 RGB levels. With relative samples the same kernel reproduces that reference exactly. The fix costs one addition per iteration to rebuild Z_m, measured at 8% of peak throughput, and it is what lets the native path follow the browser below binary64.

4 Three execution backends

Figure 1 shows the shared pipeline. The TypeScript host owns the camera, computes the reference and selects a renderer through a small common interface.

WebGPU. The primary browser backend is a WGSL fragment shader over a fullscreen triangle, with the relative reference orbit in a read-only storage buffer and the rounded centre, scale, aspect and iteration cap in a uniform block. WebGPU maps browser workloads onto modern native GPU APIs [8], and WGSL fixes the portable shader language and its host-shareable data layout [9]. Every fragment evaluates Equation 3 in `f32`. The orbit is recomputed only when the view, the Julia parameter or the iteration cap changes, so redrawing an unchanged view costs one draw call and no CPU work.

CUDA. The native executable repeats the host-side double-double orbit computation, uploads it in the same relative form, and launches one thread per pixel in 16×16 blocks with binary64 deltas. CUDA offers native single-

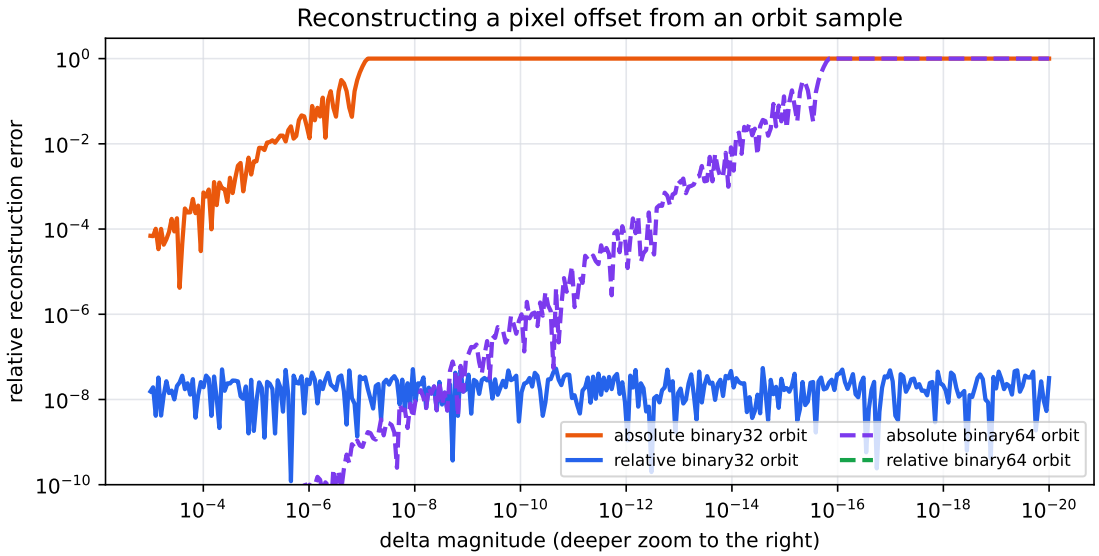

Figure 2: The mechanism, isolated: reconstructing a known pixel offset from an orbit sample stored either absolutely or relative to Z_0. The absolute form loses the offset entirely once it falls below the local spacing of the format, nine orders of magnitude apart for the two precisions; the relative form holds its relative accuracy however small the offset gets. Controlled arithmetic, not a renderer measurement – Figure 3 is the same effect on real frames.

and double-precision arithmetic, but their throughput differs sharply by device class: on compute capability 8.6, the hardware retires 128 binary32 operations per clock and per multiprocessor against 2 in binary64 [10]. The executable renders PNG files and runs the same benchmark schedule as the browser harness.

WebGL2 fallback. The compatibility backend iterates directly, with no reference orbit, using a double-single GLSL coordinate representation. That representation tops out near a view scale of 10^{-14} – and only if the driver compiles the error-free transformations as written. Some do not: the tested Windows ANGLE/D3D11 path contracts them with fast-math and discards the low word, which silently reduces the shader to plain `f32`.

Rather than document that as a caveat, the backend now measures it. A 1×1 probe shader evaluates $\textsc{TwoSum}(1, 10^{-8})$ with the operands passed in a uniform, so the expression cannot be constant-folded, and reads back whether the residual survived. When it does the zoom floor is 10^{-13}; when it does not the floor drops to 10^{-4}, which is where plain `f32` stops separating pixels in a 1080-row frame. On the test machine the probe fails, and the camera is capped accordingly.

Two further reasons keep this path out of the evidence. Its timings need a 1×1 `readPixels` to mean anything, because `gl.finish()` is not reliably blocking there. And an image that is wrong has no interesting throughput. The backend exists so that a browser without WebGPU still shows a picture.

5 Correctness protocol

5.1 Independent reference

The validation path iterates Equation 1 directly with `mpmath` at 50–90 decimal digits [11]. It contains no reference orbit, no perturbation and no rebasing; it shares with the renderers only the declared pixel mapping, the escape radius, the iteration cap and the colour function. Comparing the WGSL and CUDA ports against each other would be weaker evidence, because a shared algebraic mistake would make both agree.

Each pixel's starting point is formed as the full double-double centre plus its exact screen offset, in arbitrary precision. That is what a relative-orbit kernel actually iterates: the centre is never rounded into the pixel coordinate, only into the order-one factor of the delta recurrence. The previous release compared against a binary64-rounded start instead, which was faithful to the old absolute-orbit kernel and is meaningless below 10^{-13}, where a rounded centre collapses whole rows of pixels onto the same starting value.

Six 64×64 cases exercise different failure modes: an overview, an off-centre boundary region, a 10^{-5} view, and three views centred on a repelling fixed point of $z^2 - 0.8 + 0.156i$ specified to 75 decimal digits, at 10^{-10}, 10^{-16} and 10^{-20}. The last two sit below what binary64 pixel coordinates can address at all; they exist to test the double-double camera, not just the recurrence. For each case the script records the mean absolute RGB error, the maximum channel error, the share of identical pixels, the share whose three channels are all within four levels, the share of escaped/not-escaped flips, and SHA-256 hashes of both PNG files.

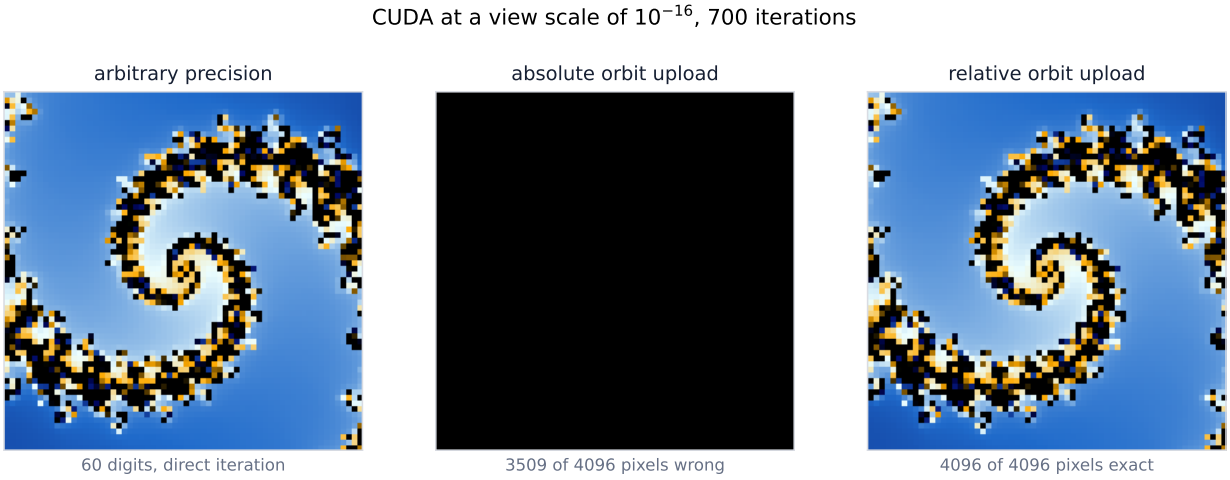

Figure 3: One line of difference in what the host uploads, four orders of magnitude below what binary64 pixel coordinates can address. The middle panel is not a failed render: it is a single colour, every pixel having collapsed onto the same quantized delta. The right-hand panel is byte-identical to the arbitrary-precision reference on all 4096 pixels.

The release gate uses three of those. A mean over 4096 pixels is far too forgiving on its own – a handful of completely wrong pixels barely moves it – so a case also fails when more than 1% of its pixels differ by more than four levels, or when more than 0.5% of them flip between escaped and not escaped. The flip count is the palette-independent one, and the one a broken rebasing step moves first.

Table 1: CUDA perturbation output against direct `mpmath` rendering, read from the archived `validation_results.csv`. Errors are 8-bit RGB levels; counts are out of the 4096 pixels of each 64×64 case. *Flips* counts pixels the two implementations disagree about escaping at all, the one column a palette cannot influence.

Case	Scale	Iter. cap	Mean error	Max error	Pixels differing	By > 4	Flips
Overview	3	300	0.000163	1	2	0	0
Off-centre boundary	3×10^{-1}	1100	0.169515	249	24	11	1
Deep zoom	10^{-5}	700	0.000163	1	2	0	0
Repelling fixed point	10^{-10}	400	0.000000	0	0	0	0
Below binary64	10^{-16}	700	0.000000	0	0	0	0
Deepest validated	10^{-20}	1200	0.144206	251	20	9	2

5.2 What the disagreements are

Two cases agree exactly: the 10^{-10} and 10^{-16} fixed-point views reproduce the arbitrary-precision render pixel for pixel. The two shallow cases are as close as an 8-bit output allows, 2 pixels of 4096 differing by a single level. The error that remains is concentrated in the off-centre case and in the deepest one, and the maxima there – 249 and 251 levels – are worth explaining rather than burying.

Every pixel that differs by more than four levels sits in a chaotic part of the escape-time map. In the off-centre case, the 3×3 neighbourhood of each such pixel spans between 524 and 1086 iterations. There, a difference of one binary64 ulp on the delta – around 10^{-16} relative – is amplified by a thousand squarings and shifts the escape time by tens of iterations, and the cyclic palette turns that shift into a distant colour. The extreme pixel, at row 60 and column 43, escapes at step 1068 under direct iteration and does not escape at all before the cap of 1100 in the CUDA render, so one image colours it and the other paints it interior black.

The shape of the error matters more than its maximum. A systematic precision fault moves many pixels a little; these cases move a few pixels a lot, with frame means of 0.17 and 0.14 levels over 4096 pixels. That is chaotic sensitivity at the boundary, not a broken recurrence – and it is why the gate reads pixel counts and flips rather than a mean.

5.3 Where the pipeline stops

The matrix says the renderer is right at 10^{-20}. It does not say where it stops being right, and that question decides what the camera should allow. The same fixed-point case, rendered at 1200 iterations from 10^{-16} to 10^{-28}, answers it (Figure 4): exact at 10^{-16}, 20 pixels differing at 10^{-20}, 2.8% of pixels differing and 0.46% flipping at 10^{-24}, and 21% differing with a mean of 4.7 levels at 10^{-28}.

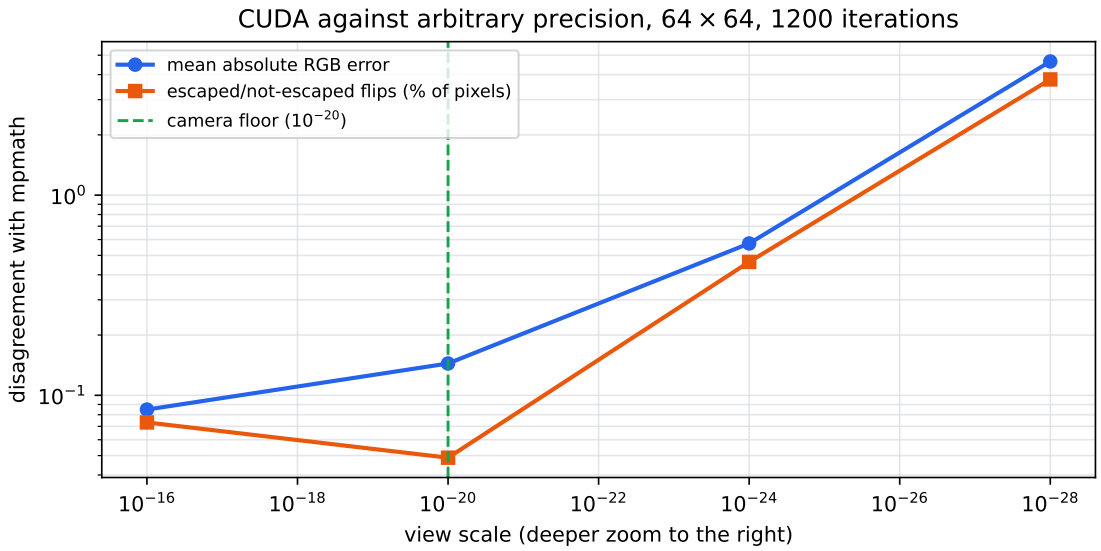

Figure 4: Archived depth sweep. Disagreement with arbitrary precision is flat until 10^{-20} and then climbs by two orders of magnitude over the next eight. The escape flips are palette-independent; the mean is not.

The mechanism is the reference orbit itself. The double-double centre carries about 31 digits, and the orbit computed from it inherits that error; once the pixel grid at a given scale is finer than the noise floor of the orbit, the perturbation has nothing left to resolve. So 10^{-28}, the figure the camera used to allow and this report used to quote, is the reach of the representation and not of the rendering. The camera now stops at 10^{-20}, the deepest scale in the frozen matrix, and one constant holds that number for both the camera and the perturbation backends.

5.4 The browser path

Everything above measures CUDA. The browser shares the TypeScript double-double, the relative-orbit preparation and the colour function, but advances its delta in binary32, so nothing here transfers to it by argument. It is now measured directly: the page opened at `/?validate` re-renders the six frozen cases with the WebGPU backend, reads the pixels back from an off-screen target, and compares them with the same archived references. The run recorded here used the same adapter as everything else in this report but a different browser build – Chromium 148 rather than the Chrome 150 of the throughput measurements – which the archived CSV header states.

The result is honest rather than flattering. The binary32 path never matches exactly, and two separate effects are visible. Almost every pixel differs by a level or two because the shaders sample the palette through a 256-entry lookup table with linear interpolation while the Python reference evaluates the gradient analytically; that is a colouring artefact, and no amount of precision removes it. The dynamics show up in the flips: 0% of pixels change escape status in the 10^{-5} and 10^{-10} cases, 0.3% in the overview, 1.0% at 10^{-16} and 2.1% at 10^{-20}. The browser is the fast path and CUDA is the accurate one, which is what the precision choice already implied but nothing had previously shown.

6 Performance experiment

6.1 Method

The benchmark sweeps 13 view scales 3×10^{-k}, $k = 0, \ldots, 12$, at 1920×1080, centred on the boundary point $0.76 + 0.24i$ so the reference orbits are long. The iteration cap is $\min(300 + 800k, 4000)$. Both harnesses use that schedule and the same throughput convention, and both exclude the reference-orbit preparation from the timed region; the timing methods differ and the difference is not negligible:

- The browser renders into an off-screen texture, with no canvas presentation and no `requestAnimationFrame`. Each depth runs three warm-up frames and reports the median of five timed frames, each bracketed by `performance.now()` around a submission followed by `GPUQueue.onSubmittedWorkDone`. Command encoding and submission overhead are therefore inside the measurement. A mean-luma readback after the timed frames rejects silently black output.
- CUDA records events immediately around the kernel launch, after the orbit upload, and reports the mean of three renders after one warm-up. Host-side work, allocation and the copy are outside the window.

The reported iteration rate is

$$R = \frac{WHI_{\max}}{t}. \tag{6}$$

It is an upper bound, since escaped pixels stop before $I_{\max}$. It stays useful for comparing backends because the schedule, the view, the escape behaviour and the counting convention are shared; how far it sits from the work actually issued is measured in Section 6.2. The NumPy baseline [12] times vectorized direct binary64 iteration at the shallow anchor only, with colouring excluded. The timed GPU kernels include their palette evaluation and their writes to the target; luma readback and file encoding stay outside.

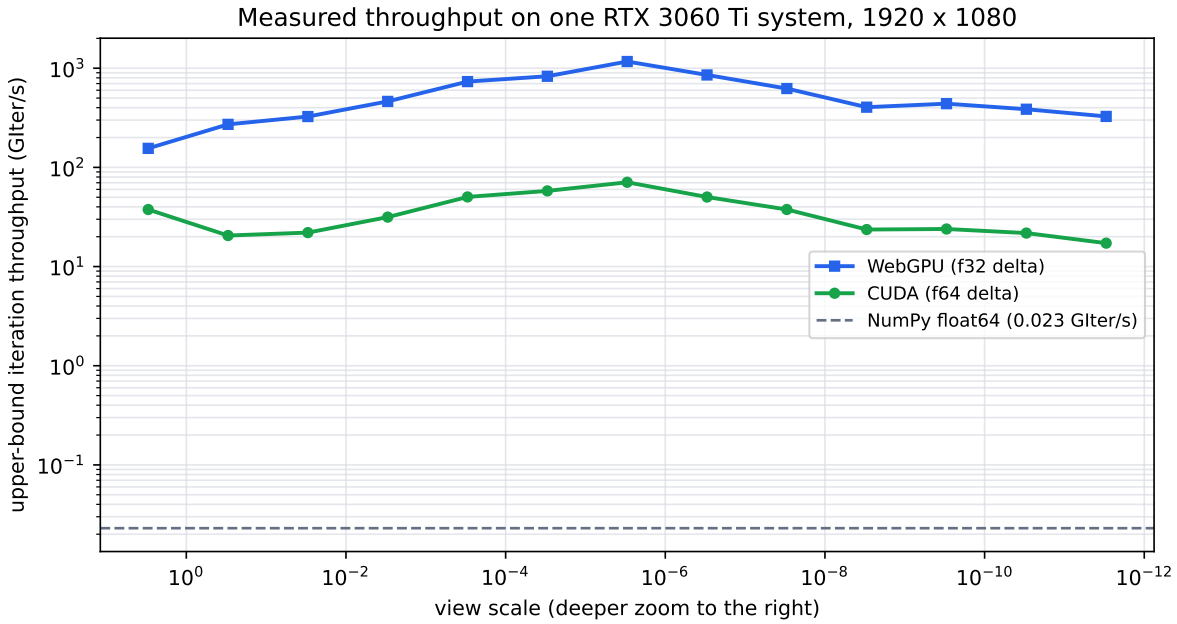

Figure 5: Frozen release measurements. The curves describe one software and hardware configuration, not WebGPU or CUDA in general. GIter/s uses the upper-bound convention in Equation 6.

Table 2: Peak upper-bound GIter/s extracted from the archived CSV files, with Mpix/s from the same row. The CPU values refer to its single shallow anchor.

Backend	Peak GIter/s	Mpix/s	Frame (ms)	Scale
WebGPU f32	1168.225	292.1	7.1	3×10^{-6}
CUDA f64	70.774	17.7	117.2	3×10^{-6}
NumPy f64	0.023	0.1	26476.6	3

6.2 Reading the numbers

Both peaks land at $k = 6$: 1168 GIter/s for WebGPU in 7.1 ms and 70.8 GIter/s for CUDA in 117 ms, a ratio of 16.5.

Before either figure is compared with anything physical, the convention needs correcting. Direct binary64 iteration over the same frame (`reference_julia.py --escape-stats`, archived as `escape_stats.csv`) shows how much of the cap the view really uses: at $k = 6$ the average pixel escapes after 159 of the 4000 permitted iterations, and the average 32-pixel group, which on a SIMT machine runs as long as its slowest lane, after 286. The machine therefore issues about 7% of the counted iterations and does about 4% of them usefully; R overstates issued work by roughly fourteen times at this point. The overstatement is not a constant either – the same measurement gives 30% at $k = 12$ – and the last paragraph of this section puts that to use.

Feeding that correction back gives a rough utilization. The inner loop is about 30 floating-point operations per iteration, counting a fused multiply-add as two. At 7% of 1168 GIter/s the WebGPU kernel issues on the order of 2.5 TFLOP/s, against the 16.2 TFLOP/s of binary32 this part provides (4864 lanes at 1.665 GHz, two flops per fused multiply-add). The CUDA kernel, at 7% of 70.8 GIter/s, issues about 150 GFLOP/s against the 253 GFLOP/s it gets in binary64 at the architectural 1:64 rate [10]. So the binary64 kernel runs near three fifths of its arithmetic ceiling while the binary32 kernel stays around a sixth of its own, and that is why the measured gap is 16.5 rather than the 64 the arithmetic ratio alone would predict. What limits the binary32 path is divergence,

control flow and buffer traffic, not precision. Perturbation is what makes that path viable at all: the extended precision is spent once, on the CPU, on a single orbit.

The NumPy anchor sits 51,000 times below the WebGPU peak, and that is the least trustworthy number here. The vectorized baseline has no early exit: it really does run all 300 iterations on every pixel, so its counted iterations are executed iterations. Correcting the GPU side alone for its 4% escape fraction brings the comparison nearer 2,000 times, and even that compares an array library against a GPU kernel rather than two serious implementations.

Finally, the curves fall with depth: WebGPU drops by a factor of 3.6 between $k = 6$ and $k = 12$. Most of that is the convention rather than the machine. Deep views escape much later – at $k = 12$ the average pixel uses 839 of the 4000 iterations and the average group 30% of the cap, against 7% at $k = 6$ – so charging both depths the full cap flatters the shallow one. Scaling each point by its measured group fraction leaves WebGPU at 84 GIter/s issued at $k = 6$ against 97 at $k = 12$, and CUDA at 5.1 against 5.1. Within the accuracy of that correction both kernels issue work at the same rate at both depths, and Figure 5 is largely a picture of how the escape statistics change with the view. What does change for a user is the frame itself: the deepest frame costs 25 ms in the browser and 482 ms in CUDA, whichever way the iterations are counted. Both figures are the cost of redrawing a cached view; opening a new one adds the CPU orbit and its upload, which the timed region excludes.

7 Reproducibility and limitations

The release carries the source, lock files, benchmark CSVs, validation PNGs, checksums and a figure manifest. Measurements were taken on Windows 10 with an RTX 3060 Ti (driver 610.62, compute capability 8.6), CUDA 13.0, Chrome 150, Node 24.18 and Python 3.12.5; the full record is in `paper/data/environment.json`. The commands are:

```
npm ci
npm test
npm run build
nvcc -O3 cuda/julia.cu -o cuda/julia.exe
uv --cache-dir .uv-cache run python scripts/validate_release.py
cuda/julia.exe --bench --w 1920 --h 1080 --out paper/data/benchmark/cuda_bench.csv
uv --cache-dir .uv-cache run python scripts/reference_julia.py --bench
Copy-Item artifacts/cpu_bench.csv paper/data/benchmark/cpu_bench.csv
uv --cache-dir .uv-cache run python scripts/reference_julia.py --escape-stats
uv --cache-dir .uv-cache run python scripts/validate_release.py --depth-sweep
uv --cache-dir .uv-cache run python paper/generate_figures.py
latexmk -cd -norc -pdf paper/main.tex
```

The browser contributes two artefacts of its own: `/?bench=webgpu` writes the throughput CSV, `/?validate` the WebGPU-versus-reference comparison, and both also emit a machine-readable record to the developer console. Figures are generated without an embedded creation date, so re-running the generator on unchanged inputs reproduces the hashes in `paper/generated/manifest.json` exactly; that is the cheapest way to check that the plotted data is the archived data. Continuous integration builds the app, runs the unit tests – including one that reads all four implementations and asserts their shared constants still agree – and runs a GPU-free self-check of the reference renderer. The PDF is rebuilt with TeX Live 2026. The software is licensed under Apache-2.0, this report and its original figures under CC BY 4.0.

What the evidence does not cover:

- one machine. Every timing comes from a single RTX 3060 Ti with one driver and one browser build. Nothing here ranks WebGPU against CUDA in general.
- RGB comparison. Two different iteration states can map to the same colour, and the browser's palette lookup table adds a level or two of its own. Comparing raw escape times would separate colouring from dynamics; the interior-flip metric is the current stand-in.
- six views, one Julia parameter. Agreement at six fixed cases with $c = -0.8 + 0.156i$, five of them around the same fixed point, is not a proof that the rebasing rule is correct for every parameter and every view.
- resolution. Every validated case is 64×64. The escape-time boundary is denser in a full frame, and the browser's flip rate would be worth re-measuring there.
- depth. The camera stops at 10^{-20} because that is where the evidence stops (Section 5.3), and the double-double representation itself gives out a few orders of magnitude below. Deeper work needs quad-double or a big-float camera.
- WebGL2. The fallback is a compatibility path, capped at 10^{-13} or, on drivers that discard the compensated arithmetic, 10^{-4}. It is outside the validated pipeline.

8 Conclusion

The division of labour is the point: one reference orbit at extended precision on the CPU, then a cheap per-pixel delta everywhere else. The recurrence itself is four lines in any language. The work is in keeping a small delta meaningful across every representation boundary it crosses, and the boundary that cost the most was the upload format. Storing $Z_m - Z_0$ instead of Z_m makes the rebasing step scale with the view; it is worth roughly eight orders of magnitude of usable depth in binary32 and three in binary64, for one addition per iteration.

The same discipline applies to the claims. Two of the numbers this report quoted in its previous version – a 10^{-28} camera and a validated renderer – turned out to describe a representation and a single backend rather than a measurement. Both are now measured: the depth sweep says where agreement ends, and the browser is compared against the same references as the native path, where it comes out looser. The next steps follow from what the measurements cannot yet separate: compare escape times rather than colours, validate at full frame resolution rather than 64×64, and run the matrix on a second device. Series approximation, progressive tiling and a quad-double camera are further out.

References

[1] John Milnor. *Dynamics in One Complex Variable*. Princeton University Press, 3 edition, 2006. DOI: 10.1515/9781400835539.

[2] K. I. Martin. SuperFractalThing maths. https://dirkwhoffmann.github.io/DeepDrill/docs/_downloads/be97cde12ee907c901e6ae8946843d94/sft_maths.pdf, 2013. Technical description of perturbation and series approximation; accessed 2026-07-28.

[3] Claude Heiland-Allen. Perturbation techniques applied to the Mandelbrot set. Technical report, mathr, 2013. https://mathr.co.uk/mandelbrot/perturbation.pdf, accessed 2026-07-28.

[4] Claude Heiland-Allen. Deep zoom theory and practice (again). https://www.mathr.co.uk/blog/2022-02-21_deep_zoom_theory_and_practice_again.html, 2022. Discussion of Zhuoran's rebasing method; accessed 2026-07-28.

[5] Theodorus J. Dekker. A floating-point technique for extending the available precision. *Numerische Mathematik*, 18:224–242, 1971. DOI: 10.1007/BF01397083.

[6] Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3 edition, 1998.

[7] Yozo Hida, Xiaoye S. Li, and David H. Bailey. Algorithms for quad-double precision floating point arithmetic. Technical Report LBNL-46996, Lawrence Berkeley National Laboratory, 2000. https://escholarship.org/uc/item/69q5t2mj, accessed 2026-07-28.

[8] W3C GPU for the Web Working Group. *WebGPU*. World Wide Web Consortium, 2026. https://www.w3.org/TR/webgpu/, accessed 2026-07-28.

[9] W3C GPU for the Web Working Group. *WebGPU Shading Language*. World Wide Web Consortium, 2026. https://www.w3.org/TR/WGSL/, accessed 2026-07-28.

[10] NVIDIA Corporation. *CUDA C++ Programming Guide, Release 13.0*. NVIDIA Corporation, 2025. https://docs.nvidia.com/cuda/archive/13.0.0/cuda-c-programming-guide/index.html, accessed 2026-07-28.

[11] The mpmath development team. *mpmath: A Python Library for Arbitrary-Precision Floating-Point Arithmetic, Version 1.4.1*, 2026. https://mpmath.org/, accessed 2026-07-28.

[12] Charles R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. DOI: 10.1038/s41586-020-2649-2.